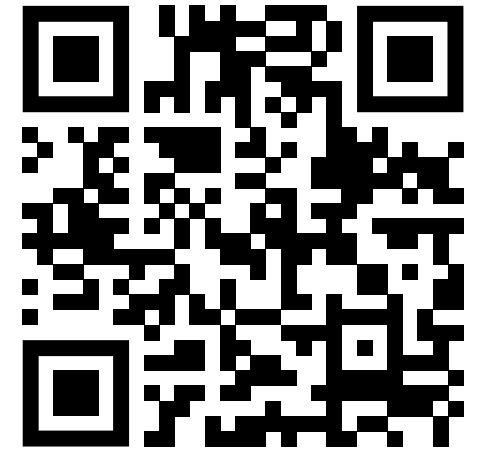


Programmierung in C++

Funktionale Programmierung

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Dan Eisenkrämer



<https://poll.hs-kempten.de/poll/>



Funktionszeiger

Lernziele

- **Funktionszeiger** können deklariert und zum flexiblen Referenzieren unterschiedlicher Funktionen verwendet werden.
 - Die Besonderheiten von Funktionszeigern in Klassen sind bekannt
 - Der Funktions-Wrapper `std::function` kann verwendet werden

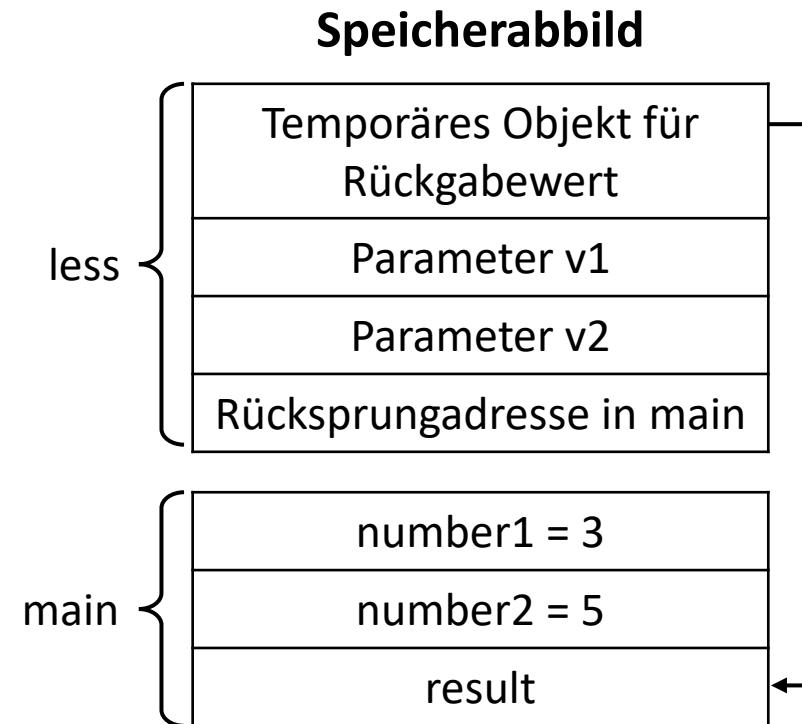
Funktionszeiger



Speicherabbild von Funktionen

- Jede Funktion wird beim Laden in den Speicher an eine bestimmte Adresse abgelegt. Im Programm wird diese Adresse durch den Funktionsnamen symbolisiert.

```
bool less(int v1, int v2) {  
    return v1 < v2;  
}  
int main() {  
    const int number1{ 3 };  
    const int number2{ 5 };  
    bool result = less(number1, number2);  
}
```

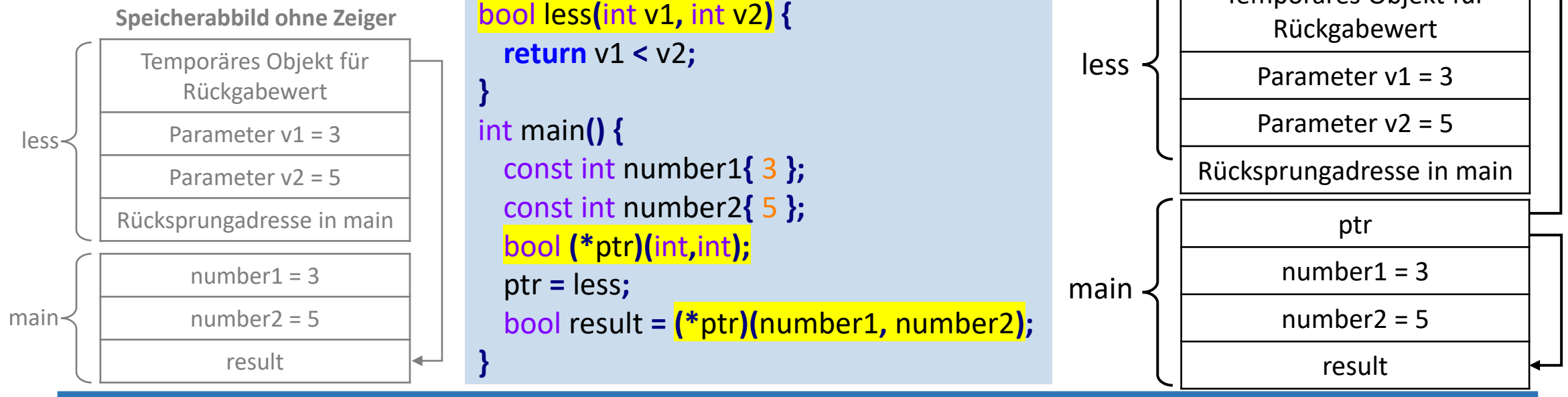


Funktionszeiger



Speicherabbild von Funktionszeigern

- Mithilfe eines **Zeigers** kann ein Verweis auf die Adresse der Funktion erstellt werden.
 - Funktionszeiger finden u.a. Einsatz im Compilerbau, bei universellen Sortialgorithmen und in der numerischen Mathematik
 - Ein Funktionsaufruf via Zeiger findet durch die Dereferenzierung des Zeigers und der Angabe der Argumente statt.



Funktionszeiger



Beispiel

- Funktionszeiger können als **Funktionsparameter** verwendet werden
 - Ermöglicht, flexibel unterschiedliche Funktionalitäten auszuführen

```
bool greater(int v1, int v2) {  
    return v1 > v2;  
}  
bool equal(int v1, int v2) {  
    return v1 == v2;  
}  
bool less(int v1, int v2) {  
    return v1 < v2;  
}
```

```
void compare(int v1, int v2, bool (*ptr)(int, int)) {  
    std::cout << std::boolalpha << (*ptr)(v1, v2) << '\n';  
}  
  
int main() {  
    const int number1{ 3 };  
    const int number2{ 5 };  
    compare(3, 5, greater);  
    compare(3, 5, equal);  
    compare(3, 5, less);  
}
```

```
false  
false  
true
```

Funktionszeiger

Deklaration



- Die Deklaration eines Funktionszeigers gleicht der Deklaration einer Funktion
 - Der Name des Zeigers ist frei wählbar
 - Damit es sich um einen Funktionszeiger und nicht um einen Datentyp-Zeiger handelt, benötigt ein Funktionszeiger Klammern um das Sternchen und seinen Namen
 - Ein Funktionszeiger kann auf eine beliebige Funktion mit den entsprechenden Eigenschaften zeigen und auch ausgetauscht werden.

Datentyp (*pointerName) (Argument1, Argument2, ..., ArgumentN)

Funktionszeiger Beispiel	Bedeutung
<code>void (*funcPtr) ()</code>	zeigt auf Funktion ohne Parameter und ohne Rückgabewerte.
<code>void (*funcPtr) (int,int)</code>	zeigt auf Funktion mit zwei Integer-Parameter und gibt keinen Wert zurück.
<code>double (*funcPtr) (double,int)</code>	zeigt auf Funktion mit je einem Parameter vom Typ double und int. Gibt einen double-Wert zurück.
<code>bool (*funcPtr) ()</code>	zeigt auf Funktion ohne Parameter und gibt einen booleschen Wert zurück.

Funktionszeiger



Template-Parametrisierung

- Funktionszeiger können wie normale Funktionen template-parametrisiert werden
 - Im Beispiel sollen die Werte eines Containers gezählt werden, die kleiner als ein gegebener Vergleichswert sind.

```
template<typename T>
bool less(T v1, T v2) {
    return v1 < v2;
}
```

```
template<typename C, typename T>
int count(const C& c, bool (*pred)(T,T), const T compValue) {
    int cnt = 0;
    for (const auto& x : c)
        if ((*pred)(x, compValue))
            ++cnt;
    return cnt;
}
```

```
void f(const Vector<int>& vec, int compValue) {
    std::cout << "Number of values less than "
    << x << ": " << count(vec, less<int>, compValue) << '\n';
}
```

count erwartet einen Funktionszeiger auf eine Vergleichsfunktion, die zwei Parameter vom Typ T erhält und einen booleschen Wert zurückgibt.

Jedes Element x aus dem Container soll mit dem compValue verglichen werden.

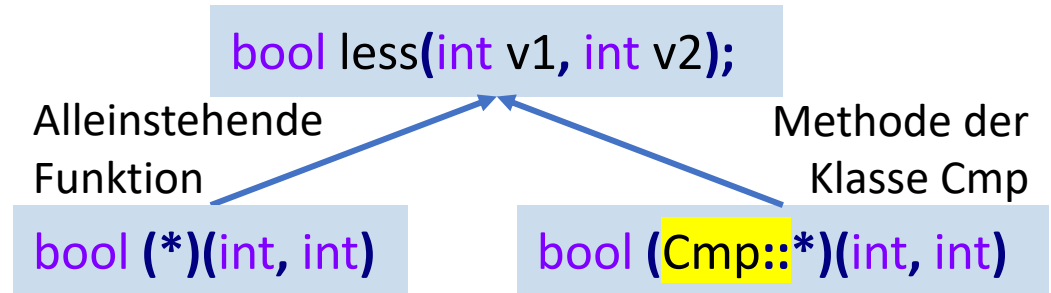
Statt less können weitere Funktionen definiert werden, die einen anderen Vergleich implementieren.

Funktionszeiger

Zeiger auf Member-Funktionen



- Der Typ von alleinstehenden Funktionen und **nicht-statischen Methoden** ist **unterschiedlich**
 - Der Typ einer statischen Member-Funktion ist wieder gleich einer alleinstehenden Funktion
- Mithilfe von **typedef** werden Funktionszeiger auf Methoden umgänglicher
 - Dadurch muss der Zeiger nur einmal definiert werden und danach kann sein Name verwendet werden
 - Um einen Funktionszeiger auf Methoden zu verwenden, muss der Zeiger auf eine konkrete Instanz angewendet werden



```
typedef bool (Cmp::*CmpMemFnc)(int v1, int v2);
```

```
int main() {  
    // Pointer auf Member-Funktion  
    CmpMemFnc ptr = &Cmp::less;  
  
    // Erzeuge Instanz von Cmp  
    Cmp x;  
  
    // Wende Funktionszeiger an  
    bool result = (x.*ptr)(3,5);  
}
```

```
class Cmp {  
public:  
    bool greater(int v1, int v2);  
    bool equal(int v1, int v2);  
    bool less(int v1, int v2);  
    // ...  
};
```

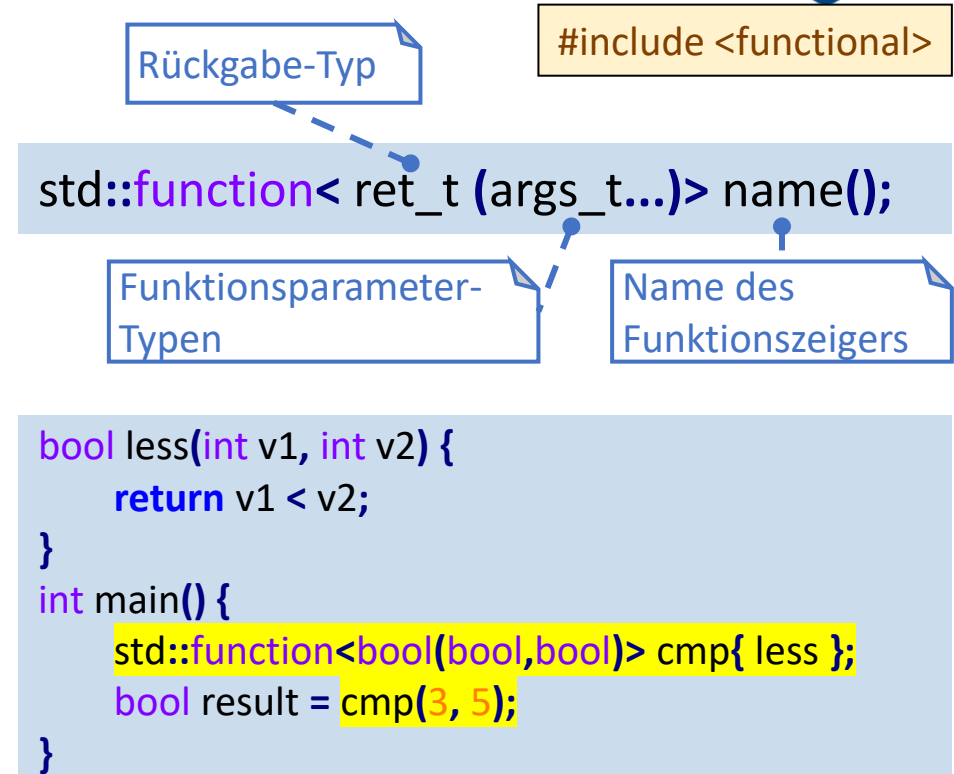

Funktionszeiger

C++11 `std::function`



- `std::function()` ist eine Funktionswrapperklasse
 - Kann jedes aufrufbare Objekt speichern und aufrufen
 - Nutzt Template-Parameter für die Funktionsinformationen
 - Für Zeiger auf Member-Funktionen wird die Instanz als erster Parameter übergeben

```
int main() {  
    // Pointer auf Member-Funktion  
    std::function<bool(Cmp&, bool, bool)> ptr  
        = &Cmp::less;  
    // Erzeuge Instanz von Cmp  
    Cmp x;  
    // Wende Funktionszeiger an  
    bool result = ptr(x, 3, 5);  
}
```



Type-Erasure Idiom:

`std::function` akzeptiert alle verschiedenen funktionsähnlichen Typen und verbirgt ihren konkreten Typ. Die Typ-Sicherheit bleibt gewährleistet.

Funktionszeiger

C++11 `std::bind`



```
#include <functional>
```

- `std::bind()` erzeugt eine Funktionswrapper (`std::function`) für eine Funktion und **bindet Funktionsargumente an den Aufruf**

Typ der Funktion

Typen der Argumente

Rückgabe-Typ der Funktion

```
template< class F, class... Args >  
/* unspecified */ bind( F&& f, Args&&... args );
```

```
template< class R, class F, class... Args >  
/* unspecified */ bind( F&& f, Args&&... args );
```

Zu bindende Funktion

Zu bindende Argumente

```
void show_text(const std::string& t) {  
    std::cout << "TEXT: " << t;  
}
```

```
std::function<void ()> f = std::bind(show_text, "Bound function");  
f();
```

Obwohl `show_text()` einen Parameter besitzt, kann es dank `std::bind` an einen Funktionszeiger ohne Parameter gebunden werden.

Das Argument wird fest mit der Funktion verknüpft.

Der Funktionsaufruf findet parameterlos statt.

TEXT: Bound function

Funktionszeiger



C++11 `std::bind` – Platzhalter

```
#include <functional>
```

- Dank **Platzhalter** können über `std::bind` gebundene Argumente auch erst **zur Laufzeit** konkretisiert werden
 - Der namespace **`std::placeholders`** kennzeichnet diese Platzhalter mit einem Unterstrich als Präfix und der Nummer des Arguments (z. B. `_1`)
 - Die Reihenfolge der Argumente darf dabei geändert werden

```
using namespace std::placeholders;
```

```
auto x = std::bind(show, _1, _2, _3);
```

```
auto y = std::bind(show, _3, _1, _2);
```

```
auto z = std::bind(show, "hello", _2, _1);
```

```
void show(const std::string& a, const std::string& b, const std::string& c) {  
    std::cout << a << "; " << b << "; " << c << '\n';  
}
```

Erzeugt `std::function<void(string, string, string)>`

Erzeugt `std::function<void(string, string)>`

```
x("one", "two", "three");  
y("one", "two", "three");  
z("one", "two");
```

```
one; two; three  
three; one; two  
hello; two; one
```

Funktionszeiger

Zusammenfassung



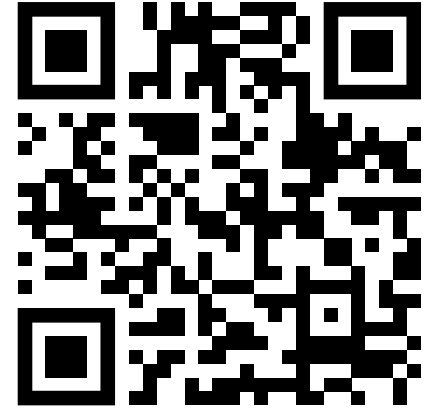
- **Funktionszeiger** sind Referenzen auf Funktionen und funktionieren wie gewöhnliche Pointer.
 - Sie können als Argumente an andere Funktionen übergeben werden.
 - Ermöglichen, flexibel unterschiedliche Funktionalitäten auszuführen.
 - **Zeiger auf Member-Funktionen** benötigen den Klassennamen und müssen auf einer Instanz aufgerufen werden.
 - **typedef** ermöglicht einen geschickteren und einfacheren Umgang mit Member-Funktionszeigern
- Die Standardbibliothek stellt mit **std::function** einen **generalisierten Wrapper für Funktionszeiger** zur Verfügung
 - Über **std::bind** werden Argumente an die Funktion gebunden

Funktionszeiger

Verständnisfragen

PIN: 681743

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Funktionszeiger können nicht auf Member-Funktionen zeigen.
- Mit `std::function` werden Template-Funktionen deklariert.
- Funktionszeiger können als Parameter an Funktionen übergeben werden.

■ Welche der folgenden Codezeilen sind gültige Funktionszeiger-Deklarationen?

- `int (*funcPtr)(int, char);`
- `decltype(int (*funcPtr)(int)) funcPtr(int);`
- `void (*funcPtr)(int, char);`
- `int (Cls::*funcPtr)(int, char);`
- `double (*funcPtr)(const std::string&);`
- `int* funcPtr[5];`
- `int* (*funcPtr)();`
- `char std::unique_ptr<funcPtr>(int);`
- `(void*) funcPtr();`



Funktionsobjekte

Lernziele

- Das Konzept von **Funktionsobjekten** wurde verstanden und sie können als Argument für Template-Parameter verwendet werden.
- Der Zusammenhang zwischen Funktionsobjekten und **Lambda Ausdrücken** wurde verstanden.
 - Der Umgang mit Lambda Ausdrücken wird beherrscht.

Funktionsobjekte

Nutzen & Aufbau



- Ein Funktionsobjekt definiert Objekte, die wie Funktionen aufgerufen werden können.
 - Dazu muss der ***operator()*** überladen werden
 - Funktionsobjekte beinhalten die Werte, die für den Aufruf relevant sind, sodass nicht für jeden möglichen Wert (und jeden möglichen Typen) eine eigene Funktion geschrieben werden muss.

```
template<typename T>
class Less_than {
    const T m_value; // Vergleichswert
public:
    Less_than(const T& value) : m_value(value) {}
    // Aufrufsoperator
    bool operator()(const T& a) const {
        return a < m_value;
    }
};
```

```
// lessI vergleicht i mit 42 via i < 42
Less_than lessI{42};
// lessS vergleicht s mit "Backus" via s < "Backus"
Less_than lessS{"Backus"s};
int main() {
    bool b1 = lessI(30);    // TRUE, da 30 < 42
    bool b2 = lessI(42);    // FALSE, da 42 == 42
    bool b3 = lessS("Naur"); // FALSE, da 'N' > 'B'
}
```

Funktionsobjekte

Nutzen & Aufbau



- Ein Funktionsobjekt definiert Objekte, die wie Funktionen aufgerufen werden können.
 - Dazu muss der *operator()* überladen werden
 - Funktionsobjekte beinhalten die Werte, die für den Aufruf relevant sind, sodass nicht für jeden möglichen Wert (und jeden möglichen Typen) eine eigene Funktion geschrieben werden muss.
 - Funktionsobjekte werden häufig als Argumente für verschiedenen Algorithmen verwendet
 - Ein Funktionsobjekt, dass beim Aufruf einen booleschen Wert zurückgibt, heißt „**predicate**“

```
template<typename C, typename P>
// C soll ein Container sein, P ein Predicate
int count(const C& c, P pred) {
    int cnt = 0;
    for (const auto& x : c)
        if (pred(x)) // Zählt alle Elemente, die "pred" erfüllen
            ++cnt;
    return cnt;
}
```

Der Vergleichswert muss nun nicht mehr übergeben werden.

```
void f(const Vector<int>& vec, int compValue) {
    std::cout << "Number of values less than "
    << x << ": "
    << count(vec, Less_than{compValue}) << '\n';
}
```

Less_than{x} erstellt ein Objekt vom Typ Less_than<int> zum Vergleich mit compValue.

Funktionsobjekte



C++11 Lambda Ausdrücke – Motivation

- Manchmal benötigen wir Funktionen nur einmalig für eine kleine Funktionalität
 - Es kann umständlich sein, Funktionsobjekte und Funktionen an einem bestimmten Ort zu definieren, um sie dann einmalig an einem anderen Ort zu verwenden.

Das Funktionsobjekt befindet sich an ganz anderer Stelle als der Aufruf.

```
template<typename T>
class Less_than {
    const T m_value; // Vergleichswert
public:
    Less_than(const T& value) : m_value(value) {}
    // Aufrufsoperator
    bool operator()(const T& a) const {
        return a < m_value;
    }
};
```

```
void f(const Vector<int>& vec, int compValue) {
    std::cout << "Number of values less than "
    << x << ": "
    << count(vec, Less_than{compValue}) << '\n';
}
```

Für einen kleinen Aufruf wirkt die Implementierung eines Funktionsobjektes unnötig aufwendig.

Funktionsobjekte



C++11 Lambda Ausdrücke – Motivation

- Was benötigen wir eigentlich?
 - a) Einen aus dem Kontext gegebenen Vergleichswert **compValue**.
 - Wir wollen mit **[]** kennzeichnen, dass die Variable aus dem Kontext zu entnehmen ist.
 - b) Einen Funktionsaufruf mit einem **Parameter a**.
 - Funktionsparameter sollen wie bei Funktionsaufrufen in **()** gekennzeichnet sein.
 - c) Eine **Vergleichsfunktion** für **a < compValue**.
 - Definitionen sind wir innerhalb von geschweiften Klammern **{ }** gewohnt.
- Was brauchen wir nicht?
 - Eine Klasse
 - Einen Funktionsnamen
 - Eine Trennung der Funktionalität

```
template<typename T>
class Less_than {
    const T m_value;
public:
    Less_than(const T& value)
        : m_value(value) {}
    // Aufrufsoperator
    bool operator()(const T& a) const
    { return a < m_value; }
};
```

a) compValue

b) Parameter a

c) Vergleichsfunktion

```
void f(const Vector<int>& vec, int compValue) {
    std::cout << "Number of values less than "
    << x << ": " << count(vec,
        [compValue] (int a) { return a < compValue; }
    ) << '\n';
}
```

a) []

b) ()

c) {}

Funktionsobjekte



C++11 Lambda Ausdrücke – Übersicht

- **Lambda Ausdrücke** erzeugen ein **namensloses Funktionsobjekt** implizit an der entsprechenden Stelle im Code.
 - Eignen sich, wenn eine spezielle Funktionalität nur an dieser Stelle im Code benötigt wird

```
void f(const Vector<int>& vec, int compValue) {  
    std::cout << "Number of values less than "  
        << x << ": "  
        << count(vec, [&](int a) {return a < compValue; }) << '\n';  
}
```

Deklarator

Aufnahmeklausel:

Gekennzeichnet durch die eckigen Klammern. Gibt an, welche Variablen auf welche Art an den Funktionskörper weitergegeben werden.

Funktionskörper:

Definition des Funktionsverhaltens.

Funktionsparameter:

Parameter, mit denen die Lambda-Funktion aufgerufen wird.

Die Lambdafunktion kann genau wie Funktionsobjekte an eine Template-Funktion übergeben werden.

```
template<typename C, typename P>  
int count(const C& c, P pred) {  
    int cnt = 0;  
    for (const auto& x : c)  
        if (pred(x))  
            ++cnt;  
    return cnt;  
}
```

Funktionsobjekte



C++11 Lambda Ausdrücke – Aufnahmeklausel

- Durch die **Aufnahmeklausel** des Lambda Ausdruck werden die Variablen an das Funktionsobjekt übergeben, die es sich abspeichert.
 - **[]** fängt keine Variablen: Es wird nichts an den Funktionskörper übergeben
 - **[&]** fängt alle lokalen Variablen im einschließenden Bereich und übergibt sie als Referenz an den Funktionskörper
 - **[=]** fängt alle lokalen Variablen im einschließenden Bereich und übergibt eine Kopie ihrer Werte an den Funktionskörper
 - **[x, &y]** übergibt die Variable x als Kopie und y als Referenz an den Funktionskörper
 - **[=, &x]** übergibt alle lokalen Variablen **außer x** an den Funktionskörper, da x als Referenz übergeben wird. Ein Bezeichner kann nicht mehr als einmal in einer Aufnahmeklausel angezeigt werden, z.B. via [=,x] (= ist Default, daher ist ,x' keine Ausnahme)
 - **[args...]** fängt ein Parameterpaket und übergibt es als Kopie. Mit vorgestelltem **&** kann auch hier das Paket als Referenz übergeben werden.
 - **C++17** **[this]** fängt den Zeiger auf das aktuelle Klassenobjekt und übergibt ihn.
 - **C++17** **[*this]** erzeugt eine Kopie des aktuellen Klassenobjektes und übergibt diese.

Funktionsobjekte



C++11 Lambda Ausdrücke – Referenz vs. Kopie

Fangen via Referenz

```
[&compValue](int a) {return a < x; }
```

- Erstelltes Funktionsobjekt:

```
struct _unnamedLambda {  
    _unnamedLambda(int& compValue)  
        : value(compValue) { } // Referenz  
  
    auto operator()(int a) const {  
        return a < value;  
    }  
  
    int& value;  
};
```

Der Aufrufoperator wird
automatisch **const**
deklariert.

Fangen via Kopie

```
[compValue](int a) {return a < x; }
```

- Erstelltes Funktionsobjekt:

```
struct _unnamedLambda {  
    _unnamedLambda(int compValue)  
        : value(compValue) { } // Kopie  
  
    auto operator()(int a) const {  
        return a < value;  
    }  
  
    int value;  
};
```

Funktionsobjekte



C++11 Lambda Ausdrücke – Referenz vs. Kopie

- Soll der Wert der Variablen im Funktionsobjekt geändert werden können, gibt es wichtige Punkte zu beachten
 - Referenzerfassungen können verwendet werden, um **Variablen außerhalb des Funktionsobjektes** zu ändern. Werterfassungen können dafür nicht verwendet werden.
 - Referenzerfassungen spiegeln **Aktualisierungen von Variablen** wider, die außerhalb des Funktionsobjektes geändert werden. Werterfassungen hingegen nicht.
 - Referenzerfassungen sind in ihrer **Lebenszeit abhängig** von den Variablen, die sie referenzieren. Werterfassungen haben keine Lebenszeitabhängigkeiten.
 - Da der Aufrufoperator standardmäßig als **const** deklariert wird, können die kopierten Werte in einem Lambdalausdruck nicht verändert werden.
Wird das Schlüsselwort **mutable** angegeben, so ist der Aufrufoperator nicht mehr **const**, wodurch die kopierten Werte geändert werden können.

Funktionsobjekte



C++11 Lambda Ausdrücke – Referenz vs. Kopie

Fangen und Ändern via Referenz

```
[&x](int a) {return a < x++; }
```

- Erstelltes Funktionsobjekt:

```
struct _unnamedLambda {  
    _unnamedLambda(int& x)  
        : value(x) { } // Referenz  
    auto operator()(int a) const {  
        return a < value++;  
    }  
    int& value;  
};
```

Für die Referenzerfassung
kann der Aufrufoperator
const bleiben.

```
Vector v{ 43,40,43,45 };  
int x = 42;
```

```
std::cout << "Count: " << count(v, _unnamedLambda{x}) << "\nx: " << x;
```

Ausgabe via **Count: 2**
Referenz: **x: 46**

Fangen und Ändern via Kopie

```
[x](int a) mutable {return a < x++; }
```

- Erstelltes Funktionsobjekt:

```
struct _unnamedLambda {  
    _unnamedLambda(int x)  
        : value(x) { } // Kopie  
    auto operator()(int a) const {  
        return a < value++;  
    }  
    int value;  
};
```

Beispiel für den Unterschied zwischen Referenz und Kopie: Statt
_unnamedLambda wird jeweils der entsprechende Lambda Ausdruck eingesetzt.

Ausgabe via **Count: 2**
Kopie: **x: 42**

Funktionsobjekte



C++11 Lambda Ausdrücke – Parameterliste

- Die **Parameterliste** eines Lambda Ausdrucks (der Lambda-Deklarator) ist optional und ähnelt in den meisten Aspekten der Parameterliste für eine Funktion.
 - Werden keine Argumente an den Lambda Ausdruck übergeben **und** besitzt der Lambda-Deklarator keine weiteren Spezifikationen, können die leeren Klammern der Parameterliste weggelassen werden.
 - Bei dem Lambda-Deklarator muss genau wie bei Parameterlisten für Funktionen auf mögliche Lebenszeitabhängigkeiten geachtet werden.
 - **C++14** Wenn ein Parametertyp generisch ist, kann mit dem Schlüsselwort **auto** ein **generischer Lambda Ausdruck** erstellt werden.

```
template<typename T, typename U>
void f(Vector<T>& vec, U x) {
    std::cout << "Number of values less than "
        << x << ": "
        << count(vec, [&x](auto& a) {return a < x; })
        << '\n';
}
```

Erzeugt

```
template<typename T, typename U>
struct _unnamedLambda {
    _unnamedLambda(U& x) : value(x) {}
    auto operator()(T& a) const {
        return a < value;
    }
    U& value;
};
```


Funktionsobjekte



C++20 Lambda Ausdrücke – Explizite Template-Parameter

- Statt der Generalisierung eines Lambda-Ausdrucks mit `auto` können explizit **Template-Parameter** angegeben werden
 - Auf das Schlüsselwort `template` wird dabei verzichtet
 - Die Template-Parameterliste wird **vor** der Funktions-Parameterliste angegeben
 - ermöglicht, im Funktionskörper mit einem **Typ-Platzhalter** zu arbeiten

```
auto pow_n = [<typename T>(const T& v, int n)
{
    auto product = static_cast<T>(1);
    for (int i = 0; i < n; ++i) {
        product *= v;
    }
    return product;
};
```

Funktionsobjekte



C++11 Lambda Ausdrücke – Rückgabebetyp

- Der **Rückgabebetyp** von Lambda Ausdrücken wird automatisch hergeleitet.
 - Durch einen nachgestellten Rückgabebetyp kann der Rückgabebetyp des Lambda Ausdrucks explizit angegeben werden.
 - Der **nachgestellte Rückgabebetyp** ist zwingend erforderlich, wenn eine automatische Deduktion nicht möglich ist. Z.B. wenn der Funktionskörper aus mehr als einer einzelnen Rückgabeanweisung besteht.

```
auto x1 = [](int i) { return i; }; // OK: Rückgabewert ist int
```

```
auto x2 = [] { return{ 1, 2 }; };
```

```
// ERROR: Eine in geschweifte Klammern gesetzte Liste stellt  
// keinen Rückgabebetyp für diese Lambdafunktion bereit.
```

```
auto x3 = []() -> std::vector<int> { return{ 1, 2 }; }; • - - - - -
```

```
// OK: Rückgabewert ist std::vector<int>
```

```
auto x4 = [=]() -> std::pair<decltype(x1),decltype(x3)> • - - - - -  
    { return { x1,x3 }; };  
// OK: Rückgabewert ist std::pair<decltype(x1),decltype(x3)>
```

Wird ein Rückgabebetyp spezifiziert, muss die Parameterliste angegeben werden.

`decltype()` gibt den Datentyp des Elements zurück.

Funktionsobjekte



C++11 Lambda Ausdrücke – Anwendung: Initialisierung

- Lambda Ausdrücke können zur **Initialisierung** verwendet werden

Komplizierte „Initialisierung“ ohne Lambda

```
// Erzeuge einen Gegner abhängig von einer Nutzereingabe
char input;
std::cin >> input;
Gegner* spawnedEnemy = nullptr;
switch (input) {
case 'D':
    spawnedEnemy = new Drache("Alduin", 200);
    break;
case 'R':
    spawnedEnemy = new Ratte("Rémy", 2);
    break;
case 'O':
    spawnedEnemy = new Oger("Shrek", 10, 2);
default:
    break;
}
```

Hier findet die Initialisierung statt.

Das ist keine Initialisierung, sondern eine Zuweisung!

Initialisierung mit Lambda

```
// Erzeuge einen Gegner abhängig von einer Nutzereingabe
char input;
std::cin >> input;
auto spawnedEnemy = [&input]()->Gegner* {
    switch (input) {
    case 'D': return new Drache("Alduin", 200);
    case 'R': return new Ratte("Rémy", 2);
    case 'O': return new Oger("Shrek", 10, 2);
    default: return nullptr;
    }
}();
```

Der Lambda Ausdruck muss aufgerufen werden.

Der Rückgabetypp muss explizit angegeben werden, damit im Switch-Statement nicht versucht wird, Pointer vom Typ der unterschiedlichen Kindklassen zu erstellen.

Funktionsobjekte



C++14 Lambda Ausdrücke – Generalisierte Erfassung

- In der Aufnahmeklausel können neue Variablen eingeführt und initialisiert werden, ohne dass diese Variablen im lokalen Bereich existieren müssen.
 - Die Initialisierung kann mit einem beliebigen Ausdruck ausgedrückt werden.
 - Der Typ der neuen Variablen wird automatisch deduziert.
 - Diese Art der Aufnahmeklausel wird als **generalisierte Erfassung** bezeichnet.
 - Kann nützlich sein, wenn Objekte übergeben werden sollen, die nur verschoben und nicht kopiert werden können wie `std::unique_ptr` (was jedoch gut durchdacht sein muss!).

```
void f(const Vector<int>& vec, std::unique_ptr<int>& x) {  
    std::cout << "Number of values less than " << *x << ": "  
    << count(vec, [ptr = std::move(x)](auto& a) {return a < *ptr; }) << '\n';  
}  
int main() {  
    Vector v{ 43,40,43,45 }; auto x = std::make_unique<int>(42);  
    f(v, x);  
    std::cout << *x;  
}
```

Dereferenzierung von x funktioniert.

Generalisierte Erfassung

unique_ptr wird an ptr übergeben. x wird zum nullptr

Durch Funktion f erzeugte Ausgabe:

Number of values less than 42: 1

Dereferenzierung von x scheitert, da x ein nullptr ist!

Funktionsobjekte

Zusammenfassung



- Durch das Überladen des **Aufruf-Operators ()** kann ein Objekt wie eine Funktion verwendet werden. Diese Objekte nennt man **Funktionsobjekte**.
 - Funktionsobjekte werden als Argument für verschiedene Algorithmen verwendet.
 - Die Membervariablen eines Funktionsobjektes können für jeden Funktionsaufruf verwendet werden, ohne, dass sie erneut angegeben werden müssen.
- **Lambda-Ausdrücke** sind eine Kurzschreibweise für Funktionsobjekte, um sie implizit an einer geeigneten Stelle im Code definieren zu können.
 - Sie helfen, um die Implementierung des Ausdrucks und seine Verwendung im Code kompakt und nah beieinander zu haben.
 - Lambda-Ausdrücke können den Code sauberer und übersichtlicher machen und Fehler vorbeugen, wenn sie geschickt eingesetzt werden.

Funktionsobjekte

Verständnisfragen

PIN: 681743

<https://poll.hs-kempten.de/poll/>



- **Wie kann ein Funktionsobjekt in C++ aufgerufen werden?**
 - Durch direkten Aufruf der Funktion, die das Funktionsobjekt kapselt.
 - Durch Aufrufen des Funktionsobjekts als wäre es eine Funktion.
 - Funktionsobjekte können nicht aufgerufen werden.
 - Mit der Methode `.invoke()` des Funktionsobjekts.
- **Welche Methoden werden verwendet, um Funktionsobjekte zu erstellen?**
 - Funktionszeiger
 - Lambda-Ausdrücke
 - Makros
 - Klasse mit `operator()`
- **WAHR oder FALSCH?**
 - Funktionsobjekte werden mit dem Schlüsselwort `function` deklariert.
 - Lambda-Ausdrücke ermöglichen die Erstellung anonymer Funktionen.
 - Lambda-Ausdrücke können als Parameter an Funktionen übergeben werden.

Funktionsobjekte

Verständnisfragen



■ Welche dieser Lambda-Ausdrücke sind syntaktisch korrekt?

- `[void] () { }`
- `[&] { }`
- `[void] () -> void { }`
- `{lambda x: return x*2;}`
- `[a, b] { return a + b; }`
- `[a, b] -> int { return a + b; }`
- `[&, b] () -> int { return a + b; }`
- `[=, b] { return a + b; }`
- `[] { std::cout << "Hi\n"; }`
- `[](int x) -> int { return (double)(x / 2); }`
- `[] () -> { return 42; }`
- `[c = v] (auto& m) {return m == c;}`
- `[&<typename V>(V a, V b)->std::vector<V>{ return{1,2}; }`

Funktionale Programmierung



C++11 Lambda Ausdrücke und std::function

```
#include <functional>
```

- *std::function* kann auch Lambda Ausdrücke verwalten
 - Da Lambda ein Funktionsobjekt erzeugt, erzeugt *std::function* eine **virtuelle Funktion**, um zur Laufzeit die Funktion korrekt ausführen zu können
 - Für die Daten der Aufnahmeklausel aus dem Lambda-Ausdruck muss *std::function* **Speicherplatz dynamisch allokalieren**
 - => *std::function* mit einem Lambda Ausdruck zu verwenden, hat einen **Einfluss auf das Laufzeitverhalten**

```
std::function<void()> f_display_42 = []() { std::cout << 42 << '\n'; };  
f_display_42();
```

```
std::function<int()> f_num = []() { return 42; }
```

```
std::function<void(int)> f_display_num = [](int i) { std::cout << i << '\n'; };  
f_display(f_num());
```


Funktionale Programmierung



C++11 „Rekursive Lambda“

```
#include <functional>
```

- Lambda Ausdrücke mit *std::function* können auch für rekursive Aufrufe genutzt werden
 - Das *std::function* Objekt kann also in seiner eigenen Definition verwendet werden
 - Dazu darf jedoch nicht die automatische Typ-Deduktion verwendet werden

```
auto factorial = [](int n)
{
    // store a lambda object to emulate "recursive lambda"; aware of extra overhead
    std::function<int(int)> fac = [&](int n) { return (n < 2) ? 1 : n * fac(n - 1); };
    // note that "auto fac = [&](int n) {...};" does not work in recursive calls
    return fac(n);
};
for (int i{5}; i != 8; ++i)
    std::cout << i << "! = " << factorial(i) << " ";
```

```
5! = 120; 6! = 720; 7! = 5040;
```